

# **Guardrail: Context-Driven Security Operations Platform**

Project Workbook

By

Nikhil Dupally

Pooja Ramesh Sindham

Vijaya Sharavan Reddy Badddam

Viswa Surya Kumar Suvvada

April 9, 2026

**Advisor: Prof. Park Younghee**

# **Chapter 1. Literature Search, State of the Art**

## **1.1 LITERATURE SEARCH**

The problem of log anomaly detection is concerned with identifying unusual system behavior such as failures, misconfigurations, degradations in system performance, or unusual activity in systems and applications using log data. Log anomaly detection is a difficult problem since log data is available in huge volumes, is only semi-structured, and is subject to constant changes in software applications.

Early academic work formalized anomaly detection as a sequence learning problem. DeepLog models log data as sequences of events and uses LSTM-based predictions for identifying anomalies when sequences of events deviate from normal behavior[1]. However, advances in log anomaly detection critically depend on the availability of representative data for log sequences. To facilitate log anomaly detection using representative data, LogHub was created as a large collection of log data and a tool for reproducibility in log anomaly detection[2]. Log sparsity is a common problem in log data preprocessing. Drain is a popular online log parsing tool that is used for log sparsity reduction using a technique called clustering log messages into event templates[3].

More recent advances in log anomaly detection have focused on robustness and generalization in log anomaly detection. LogRobust is a system designed to make log anomaly detection more robust when dealing with unstable log data using a combination of semantic representations and sequence modeling[4]. LogRobust is less affected by changes in log event templates. LogAnomaly is a system for detecting both sequential and quantitative anomalies in unstructured log data. LogAnomaly is less affected by changes in log event templates using representations learned from log data[5]. LogBERT is a state-of-the-art log anomaly detection system that uses a transformer-based model for log anomaly detection using self-supervised learning objectives for normal log pattern learning and anomaly detection using BERT representations[6]. LogGPT is a state-of-the-art log anomaly detection system that uses a transformer-based model for log anomaly detection using log event representations trained using a variant of the GPT model. LogGPT also uses task-aligned model tuning for bridging the gap between log language modeling and log anomaly detection[7]. LLMeLog (ISSRE) is a state-of-the-art log anomaly detection system using LLM capabilities for deeper log event understanding for log anomaly detection[8].

## **1.2 RESEARCH PAPER INPUTS**

The first step in our Guardrail strategy is to identify peer-reviewed and popularly used academic methods for dealing with anomalies in large-scale log data. Some of these include:

- DeepLog: Develops a baseline for detecting anomalies in log data using deep models for learning normal log event sequences.
- LogHub: Assists in reproducibility in log data analysis by providing a collection of publicly available log data and standardizing evaluation conditions.
- LogRobust: Focuses on log instability and provides a method for dealing with semantic representations.
- LogAnomaly: Enhances the ability to detect both sequence and quantitative anomalies in log data and reduces false positives due to template instability.
- LogBERT: Demonstrates the use of transformer models and self-supervised learning for log data.
- LogGPT: Demonstrates the use of a GPT/LLM model for log sequences and how the training objective can be adapted for log anomaly detection.
- LLMeLog: Demonstrates state-of-the-art log anomaly detection using LLMs and enhances our understanding of log events.

The works behind Guardrail's research foundation each play a meaningful role. Together, they cover the building blocks of sequence modeling [1], the importance of reproducible benchmarking [2], and the groundwork needed for scalable preprocessing [3]. They also address how systems can stay resilient as logs evolve over time [4], sharpen the ability to detect anomalies more accurately [5], leverage transformer-based representations [6], and harness the growing power of large language models to push performance even further [7], [8].

### **1.3 STATE-OF-THE-ART SUMMARY**

In the industry, the state-of-the-art in log anomaly detection is part of an observability stack that integrates well with metrics and traces, all designed for fast response to incidents, scalability in growth, and usability in day-to-day operations. Beyond the pure offline accuracy of the detection, the industry has been focusing on the following areas: (a) detection of spikes in log volumes and unusual patterns, (b) identification of suspicious fields/messages to aid in the investigation of the anomaly, (c) reduction of false positives through the incorporation of baselines and seasonality correction, and (d) integration of the results of the anomaly detection into the overall alerting stack.

For example, the Elastic Observability stack has a “Logs Anomalies” workflow that allows the user to spin up machine learning jobs for the analysis of log rate and anomaly detection directly from the user interface, designed to assist in operation as opposed to research-based evaluations. The industry has been moving towards semantic representations, context-based models such as the “transformer” architecture, as well as the application of LLM-based analysis in the field of log anomaly detection to assist in the investigation of the anomaly results.

Guardrail targets this “leading-edge” industry direction by combining academically grounded anomaly detection methods [1]–[8] with practical design principles aligned to observability

workflows like scalable preprocessing, sequence-based classification, evaluation reporting, and outputs that support investigation and benchmarking rather than only model training.

## 1.4 REFERENCES

- [1] M. Du, F. Li, G. Zheng, and V. Srikumar, “DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2017, pp. 1285–1298, doi: 10.1145/3133956.3134015.
- [2] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, “LogHub: A Large Collection of System Log Datasets for AI-driven Log Analytics,” *arXiv preprint*, arXiv:2008.06448, 2020.
- [3] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, “Drain: An Online Log Parsing Approach with Fixed Depth Tree,” in *Proc. IEEE Int. Conf. on Web Services (ICWS)*, 2017, doi: 10.1109/ICWS.2017.13.
- [4] X. Zhang *et al.*, “Robust Log-based Anomaly Detection on Unstable Log Data,” in *Proc. ACM Joint Meeting on European Software Engineering Conf. and Symp. on the Foundations of Software Engineering (ESEC/FSE)*, 2019, doi: 10.1145/3338906.3338931.
- [5] W. Meng *et al.*, “LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs,” in *Proc. IJCAI*, 2019.
- [6] H. Guo, S. Yuan, and X. Wu, “LogBERT: Log Anomaly Detection via BERT,” in *Proc. Int. Joint Conf. on Neural Networks (IJCNN)*, 2021, pp. 1–8, doi: 10.1109/IJCNN52387.2021.9534113.
- [7] X. Han, S. Yuan, and M. Trabelsi, “LogGPT: Log Anomaly Detection via GPT,” *arXiv preprint*, arXiv:2309.14482, 2023.
- [8] M. He *et al.*, “LLMeLog: An Approach for Anomaly Detection based on Large Language Model Enriched Log Events,” in *Proc. IEEE Int. Symp. on Software Reliability Engineering (ISSRE)*, 2024.
- [9] Elastic, “Inspect log anomalies (Logs Anomalies),” Elastic Observability Documentation.

## Chapter 2. Project Justification

The Guardrail project is well justified, and the reasoning behind it is straightforward. Log data is among the earliest and most reliable signals that something has gone wrong in a modern software system, whether that is a failure, a misconfiguration, a security issue, or a decline in performance. Production environments generate logs constantly and in massive volumes, which makes them incredibly useful but also difficult to manage. Analyzing them is not a luxury; it is a fundamental part of maintaining system health and responding to problems in a timely manner.

The challenge is that manual analysis simply cannot keep up with this volume. Basic alerting systems also tend to struggle, as they either fail to catch new and unfamiliar failure patterns or produce so many false positives that engineers stop trusting them altogether.

There is obvious and useful value in a system that can automatically identify anomalous behavior in logs. It enables teams to detect issues earlier, recover more quickly, and keep their infrastructure more reliable.

Guardrail is a significant project from an academic and engineering standpoint, spanning two full semesters and situated at the intersection of software engineering and machine learning, in addition to its practical applications. Preprocessing and normalizing raw log data, creating sequences through windowing, classifying anomalies using machine learning models, and measuring performance using common evaluation metrics like accuracy, precision, recall, and F1 score are all part of the project's goal to deliver a fully functional pipeline.

The level of generality is suitable for both Part A and Part B because it covers both the essential design and implementation (data engineering, modeling, and evaluation) and the iterative process of improvement and evaluation (parameter tuning, robustness testing, and comparative testing). Moreover, the project is measurable because its performance can be evaluated using publicly available data and can be compared against other techniques in the literature as a baseline.

Guardrail has the potential to exceed expectations at the expo by providing results that can be published within the realm of applied software reliability engineering. A significant contribution to software reliability engineering can be achieved by including the following components: (a) the development of a reproducible anomaly detection workflow, (b) the evaluation of system performance using multiple benchmark datasets, and (c) the consideration of problem constraints such as robustness to changing log patterns, runtime considerations, and reporting. These components match the expectations of both industry and academic groups, as the work will be correct, reproducible, scalable, and operationally relevant. Guardrail can thus be positioned as a contribution to software reliability engineering, providing results that can be used to construct a peer-reviewed manuscript as well as a compelling expo demonstration.

## **Chapter 3. Project Requirements**

### **3.1 PURPOSE AND OBJECTIVES**

#### 3.1.1 Purpose

The primary goal of Guardrail is to build a log-driven anomaly detection system, which examines the log entries of a system and its applications, and then labels them as Normal or Anomalous to detect potential failures and misconfigurations in software systems.

#### 3.1.2 Objectives

The project objectives for building the log anomaly detection workflow include:

- a. Developing a complete workflow for data preparation, model execution, and evaluation. (data preparation → model execution → evaluation).
- b. Categorizing the log sequences into anomalous (unusual) behavior and normal behaviour.
- c. Providing the ability for log preprocessing, where variable tokens such as IDs, timestamps, and addresses can be masked/normalized.
- d. Creating labeled sequences of logs by applying windowing techniques such as sliding windowing and session windowing.
- e. Displaying evaluation results, including Accuracy, Precision, Recall, and F1-score.
- f. Providing the ability for the model to be loaded for testing.
- g. Providing the ability for the workflow to be executed without any manual intervention during execution, based on the provided configuration.

### **3.2 USE CASES / SCRUM USER STORIES**

US-1: As a user, I want the log dataset path to be provided to Guardrail to allow the ingestion of the logs for further analysis.

US-2: As a user, I want the logs to be normalized and masked to prevent the tokens of variables from impacting the robustness of the model.

US-3: As a user, I want Guardrail to produce sequences by using windowing to allow the detection of behavioral patterns in the temporal dimension.

US-4: As a user, I want Guardrail to classify the sequences as Normal or Anomalous to allow for the automatic detection of abnormal behavior.

US-5: As a user, I want Guardrail to load a model checkpoint to allow for the continuation of the inference without the need to retrain the model.

US-6: As a user, I want Guardrail to produce evaluation metrics to allow for the reporting of the results.

US-7: As a user, I want Guardrail to produce explicit error messages when the input is missing or invalid to allow for the prompt resolution of the problem.

### **3.3 SCOPE OF WORK**

Following is the scope for the project:

- a. The scope of guardrail would be to support anomaly detection over at least one publicly available log data set (e.g., HDFS, BGL, Liberty, Thunderbird).
- b. The scope would also include implementation and validation of:
  - i. Log preprocessing and normalization
  - ii. Sequence generation for training and testing
  - iii. Model inference and evaluation
- c. The classification model would be based on deep learning techniques.
- d. The log sequence would be used in the classification model instead of log statements.
- e. The predictions would be generated to be used in the final project report.

### **3.4 NON-FUNCTIONAL REQUIREMENTS**

In line with the specified purpose and objectives, the following non-functional requirements can be identified:

- a. Reproducibility: The system should be able to produce the same metrics when provided with the same set of data, checkpoint, and configuration, within a specified tolerance level.
- b. Performance/stability: The system should be able to perform batch processing, including the specification of sequence limits to prevent memory failures.
- c. Configurability: The system should allow the user to configure the path of the dataset, window size, batch size, and checkpoint path without requiring code modifications.
- d. Robustness to variability: The system should be able to reduce the impact of variable tokens to enhance robustness to log variability.
- e. Maintainability: The system should be able to perform the following as separate components with well-defined inputs and outputs: preprocessing, sequence generation, model execution, and evaluation.
- f. Usability: The system should be able to show runtime status messages to facilitate easier debugging and reporting.

### **3.5 FUNCTIONAL REQUIREMENTS**

#### **3.5.1 Essential Features**

- a. The system shall ingest log data from chosen datasets and formats through a specified input path.
- b. The system shall preprocess log data using normalization and masking rules and shall provide preprocessed log data or log sequences.
- c. The system shall generate labeled log sequences for learning and evaluation using a sliding window method and/or a session windowing method.
- d. The system shall conduct binary classification on log sequences using a Normal/Anomalous model and shall produce a predicted label for each log sequence.
- e. The system shall allow for an existing checkpoint to be loaded and test data to be processed for inference without retraining.
- f. The system shall calculate evaluation metrics for test data (Accuracy, Precision, Recall, F1 score).
- g. The system shall export results in a reusable form:

- i. a summary of metrics (console output and/or file);
  - ii. prediction results for each log sequence (file or equivalent).
- h. The system shall validate critical inputs (dataset path and checkpoint path) and terminate execution with a clear error message in case of failure.

### 3.5.2 Desired Features

- a. The system shall provide for training or fine-tuning operations, which shall be dependent on available resources, for generating a usable model checkpoint.
- b. The system shall provide for execution with a fixed random seed configuration.
- c. The system shall provide for explicit configurability of parameters without requiring changes to code.
- d. The system shall provide for basic input validation, including proper error messages for missing paths, formats, and parameters.

### 3.5.3 Optional Features

- a. The system may include support for evaluation across several datasets to enable comparison.
- b. The system may include support for generating summary reports automatically, which can include metrics tables and simple plots.
- c. The system may include support for generating limited interpretability results, which can include highlighting log lines or tokens for debugging.

## Chapter 4. Dependencies and Deliverables

### 4.1 Project Deliverables

The project deliverables are as follows for the Guardrail project:

#### 4.1.1 Technical Deliverables

- a. A functional Guardrail anomaly detection pipeline with components for preprocessing, sequence generation, inference, and evaluation.
- b. A functional module for data preprocessing and windowing that transforms raw data into model-ready sequences, with the processed data sets stored as dataset artifacts.
- c. A functional inference workflow that can load the trained model checkpoint and generate the predictions on the given test dataset.
- d. A functional evaluation workflow that can run and provide Accuracy, Precision, Recall, and F1-score for the given dataset.
- e. Model artifact used for the final evaluation (a trained checkpoint developed by the team or a validated checkpoint commonly used for the results).
- f. A results package with the prediction results and the final metrics summary table for inclusion in the final report.
- g. Architectural documentation on the system diagram with a brief data flow and model flow specific for the Guardrail project.

### 4.2 Dependencies

Following are the list of potential dependencies that could cause a project halt if they are not met:

#### 4.2.1 Data Dependencies

- a. Availability of at least one benchmark log dataset in an accessible form (i.e., raw logs or preprocessed files) with an appropriate train/test or evaluation split.
- b. Availability of ground-truth labels (or an established labeling scheme) required to evaluate anomaly detection performance.

c. Capability to generate sequences (i.e., windowing) without data corruption or leakage (i.e., logs must have appropriate fields and order).

#### 4.2.2 Compute / Runtime Dependencies

a. Availability of a working execution environment that can run the model pipeline (i.e., at least CPU if used for small-scale evaluation; GPU if training or fine-tuning is performed).

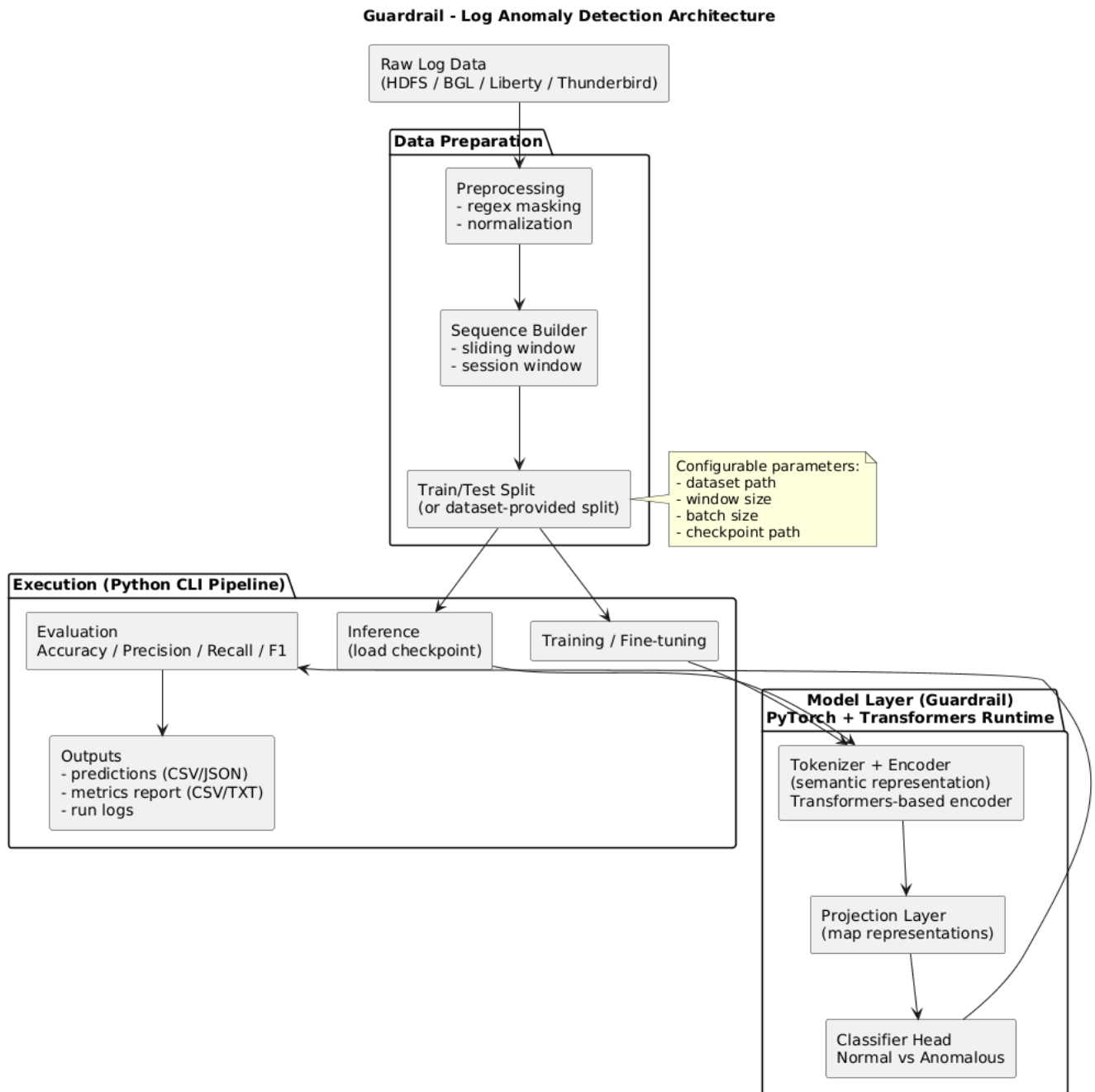
b. Sufficient storage to hold dataset files, sequences, and checkpoint data.

#### 4.2.3 Software Compatibility Dependencies

a. Availability of a working Python environment on which required ML software libraries can be installed and executed.

b. Compatibility of software library versions required to load/run checkpoints and execute inference/evaluation scripts.

## Chapter 5. Project Architecture



The Guardrail architecture has been designed to be a layered architecture for log anomaly detection in batch logs. The architecture reads raw log data (e.g., HDFS, BGL, Liberty, Thunderbird), passes it through a data preparation stage to normalize and mask noise in log data due to varying tokens like timestamps, identifiers, and addresses. The log sequence builder then uses either a sliding

window or session window to build log sequence data. The log sequence data is then divided into training and testing datasets or uses the provided splits in the datasets.

The Execution Layer has two workflows: Inference/Evaluation and optional Training/Fine-tuning. In Inference/Evaluation, the trained checkpoint is used to execute sequence classification. In Training/Fine-tuning (optional), training sequences are consumed to generate a checkpoint for Evaluation.

The Model Layer has three components: Tokenizer/Encoder, Projection Layer, and Classifier Head. The Tokenizer/Encoder uses log sequence data to generate semantic representations using a log sequence tokenizer/encoder. The Projection Layer then uses log sequence data to generate log sequence projection using log sequence projection. The Classifier Head then uses log sequence projection to generate binary class label output.

The output of the Guardrail architecture consists of a results package containing sequence level predictions and evaluation metrics like Accuracy, Precision, Recall, and F1-score.

The architecture has been designed to be modular with a layered architecture (data preparation, model layer, execution layer), which is in line with AI/SE architecture styles like Pipe and Filter architecture.

## Chapter 6. Project Design

### Chapter 6: Project Design

#### 6.1 Design Overview

The Guardrail system is designed as a **modular, extensible, and configuration-driven pipeline** that transforms raw log data into structured, actionable security insights. The design extends the architecture described in Chapter 5 by introducing additional layers for **rule-based detection and contextual reasoning**, making the system suitable for Security Operations Center (SOC) workflows.

The design follows these principles:

- **Separation of concerns:** Each stage (preprocessing, sequence modeling, detection, contextualization) is independent.
- **Pipeline-based execution:** Data flows sequentially across well-defined stages.
- **Extensibility:** New datasets, models, and rules can be added without modifying core logic.
- **Reproducibility:** Fully driven by configuration (paths, window size, checkpoints).

#### 6.2 System Design Components

The system is divided into five major design components:

##### 6.2.1 Log Ingestion and Preprocessing Module

- Input: Raw log files (HDFS, BGL, etc.)
- Operations:
  - Regex-based masking (timestamps, IPs, IDs)
  - Log normalization into templates
- Output: Cleaned and structured log events

This component ensures that the model learns **patterns instead of noise**, consistent with the preprocessing stage described in Chapter 5

##### 6.2.2 Sequence Generation Module

- Converts logs into sequences using:
  - Sliding window
  - Session window (if applicable)

- Preserves temporal ordering
- Produces labeled sequences for training/testing

This module enables **behavior-level modeling instead of log-level analysis**.

### 6.2.3 Machine Learning Detection Module

- Input: Log sequences
- Components:
  - Tokenizer / Encoder (Transformer-based)
  - Projection Layer
  - Classification Head (Normal vs Anomalous)
- Output:
  - Sequence-level anomaly predictions
  - Confidence scores (if available)

This aligns directly with the **model layer described in Chapter 5**

### 6.2.4 Rule-Based Detection Module (Extension)

This module is introduced as part of the updated design.

- Inspired by systems like:
  - Sigma
  - SNORT
- Functionality:
  - Pattern matching on logs/sequences
  - Detection of known attack signatures
  - Rule evaluation independent of ML model
- Output:
  - Rule-triggered alerts
  - Matched patterns

This enables detection of **known threats that ML may miss**.

### 6.2.5 Contextual Analysis and Output Module (New Layer)

This is the key enhancement based on your professor's feedback.

- Combines:
  - ML predictions
  - Rule-based detections
- Performs:

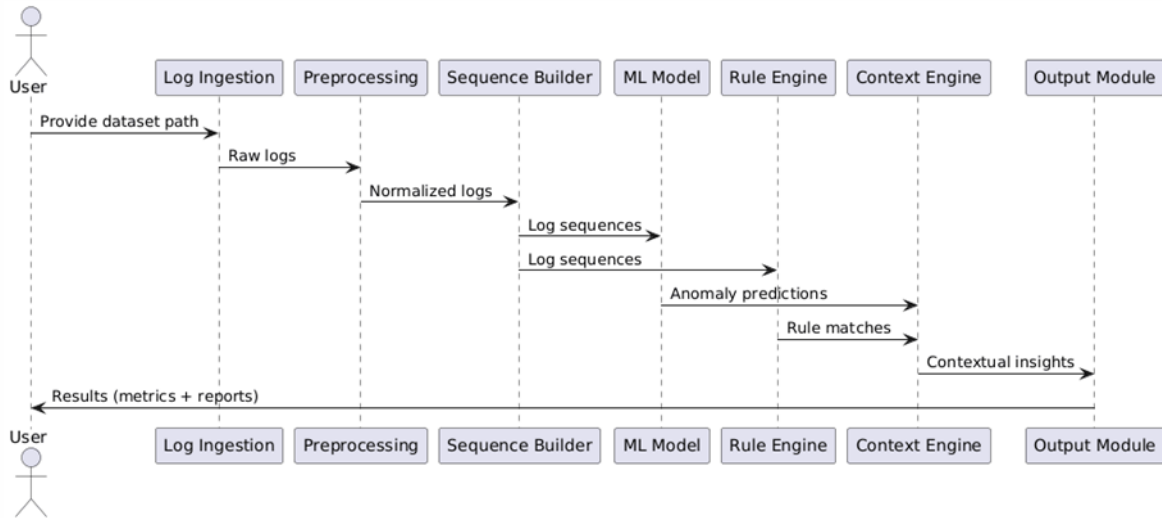
- Entity extraction (IP, user, process)
- Pattern grouping
- Explanation generation
- Produces:
  - Structured SOC-ready outputs:
    - Anomaly type
    - Supporting evidence
    - Affected entities
    - Sequence context
- Optional:
  - Mapping to MITRE ATT&CK categories

This transforms Guardrail into a **decision-support system**, not just a classifier.

### 6.3 Detailed System Flow (Sequence Design)

#### Sequence Flow Description

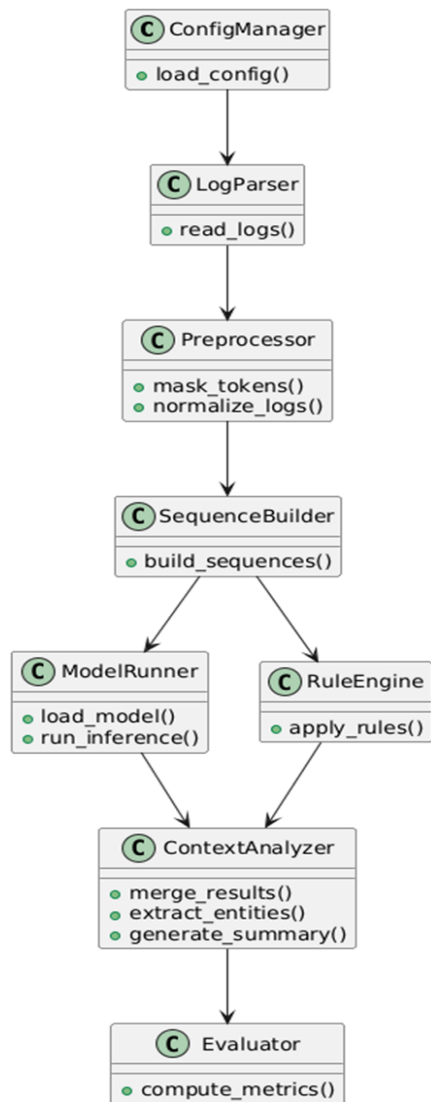
1. Logs are ingested from dataset path
2. Logs are normalized and masked
3. Sequences are generated using windowing
4. Sequences are passed to:
  - ML model → anomaly prediction
  - Rule engine → rule matches
5. Outputs are merged
6. Contextual layer generates structured insights
7. Results are exported (metrics + predictions)



## 6.4 Class Design (UML Class Diagram)

### Key Classes

- LogParser
- Preprocessor
- SequenceBuilder
- ModelRunner
- RuleEngine
- ContextAnalyzer
- Evaluator
- ConfigManager



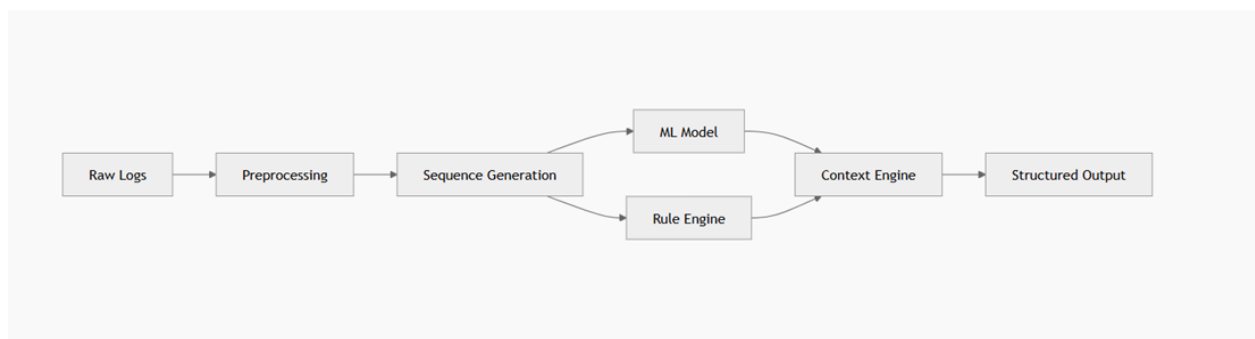
## 6.5 Data Flow Design

### Data Transformation Stages

| Stage         | Input    | Output               |
|---------------|----------|----------------------|
| Ingestion     | Raw logs | Raw log entries      |
| Preprocessing | Raw logs | Normalized templates |

|                  |             |                            |
|------------------|-------------|----------------------------|
| Sequence Builder | Templates   | Log sequences              |
| Detection        | Sequences   | Predictions + rule matches |
| Context Layer    | Predictions | Structured insights        |
| Output           | Insights    | Reports + metrics          |

Data Flow Diagram:



## 6.6 Configuration Design

The system is fully configuration-driven.

### Configurable Parameters

- Dataset path
- Window size
- Batch size
- Model checkpoint
- Rule definitions (JSON/YAML)

Example Config Structure:

dataset\_path:/data/hdfs.log

window\_size: 10

batch\_size: 32

checkpoint: /models/logbert.pt

rules\_path: /rules/rules.yaml  
output\_path: /outputs/

## **6.7 Design Trade-offs**

### **1. Sequence-based vs Log-based Analysis**

- Chosen: Sequence-based
- Reason: Captures temporal behavior and reduces false positives

### **2. ML-only vs Hybrid Detection**

- Chosen: Hybrid (ML + Rules)
- Reason:
  - ML detects unknown anomalies
  - Rules detect known attack patterns

### **3. Batch vs Real-time Processing**

- Current: Batch pipeline
- Reason: Simpler reproducibility and evaluation
- Future: Extend to streaming systems

### **4. Interpretability vs Performance**

- Added contextual layer
- Trade-off:
  - Slight overhead
  - Significant gain in usability for SOC analysts

## **6.8 Output Design**

The final output is structured and analyst-friendly:

### **Output Components**

- Prediction label (Normal / Anomalous)
- Sequence context
- Triggered rules (if any)
- Extracted entities (IP, user, process)
- Explanation summary
- Evaluation metrics:
  - Accuracy

- Precision
- Recall
- F1-score

This extends the original output design (metrics + predictions) described in Chapter 5 into a **context-aware reporting system**.

## 6.9 Summary of Design

The Guardrail design integrates:

- **Data Engineering** → preprocessing + sequence building
- **Machine Learning** → anomaly detection
- **Security Knowledge** → rule-based detection
- **Contextual Reasoning** → explainability + SOC usability

This layered and hybrid design ensures that Guardrail is:

- Technically robust
- Extensible
- Operationally relevant for real-world security workflows

## Chapter 7. QA, Performance, Deployment Plan

### 7.1 Overview

This chapter describes the quality assurance (QA), performance evaluation, and deployment strategy for the Guardrail system. The goal is to ensure that the system is **correct, reproducible, stable, and usable** for log anomaly detection tasks.

Guardrail operates as a **batch-based anomaly detection pipeline**, and therefore testing focuses on validating each stage of the pipeline, as well as the correctness of end-to-end execution.

### 7.2 Quality Assurance (QA) Plan

#### 7.2.1 Testing Strategy

The testing strategy is based on validating each major component of the pipeline:

- Data preprocessing
- Sequence generation
- Model inference
- Evaluation metrics
- End-to-end execution

Testing ensures that the system behaves correctly under expected and invalid inputs.

#### 7.2.2 Unit Testing

Unit testing focuses on validating individual modules independently.

##### Modules Tested:

1. **Preprocessing Module**
  - Verify masking of variable tokens (timestamps, IDs, IPs)
  - Ensure normalization produces consistent templates
2. **Sequence Generation Module**
  - Validate correct sequence length (window size)
  - Ensure correct ordering of logs
  - Verify no data leakage between train/test splits
3. **Model Inference Module**
  - Verify model loads correctly from checkpoint
  - Ensure predictions are generated for all sequences
4. **Evaluation Module**
  - Validate correctness of metrics:

- Accuracy
- Precision
- Recall
- F1-score

### **7.2.3 Integration Testing**

Integration testing validates the interaction between components.

#### **Test Cases:**

- Preprocessing → Sequence generation → Model inference
- Model inference → Evaluation
- Full pipeline execution with configuration file

#### **Expected Outcome:**

- Smooth data flow between modules
- No runtime errors
- Correct output format generation

### **7.2.4 End-to-End Testing**

End-to-end testing ensures that the entire system works as expected when executed as a complete pipeline.

#### **Steps:**

1. Provide dataset path and configuration
2. Run full pipeline
3. Generate predictions and metrics

#### **Validation:**

- Output files are generated correctly
- Metrics are computed successfully
- Logs show successful execution

### **7.2.5 Negative Testing**

Negative testing ensures robustness under invalid conditions.

#### **Test Scenarios:**

- Missing dataset path
- Invalid checkpoint path
- Incorrect configuration parameters
- Empty input dataset

#### **Expected Behavior:**

- System should fail fast
- Clear error messages should be displayed

### **7.3 Performance Evaluation Plan**

#### **7.3.1 Evaluation Metrics**

The performance of Guardrail is evaluated using standard classification metrics:

- **Accuracy:** Overall correctness of predictions
- **Precision:** Correctness of anomaly predictions
- **Recall:** Ability to detect actual anomalies
- **F1-score:** Balance between precision and recall

These metrics are already defined as part of the system requirements

#### **7.3.2 Baseline Evaluation**

- Use an existing dataset (e.g., HDFS or BGL)
- Run inference using a trained or provided checkpoint
- Record:
  - Prediction outputs
  - Metrics summary

This corresponds to **Phase 3 (Baseline Model Execution)** in the implementation plan

#### **7.3.3 Reproducibility Testing**

To ensure reproducibility:

- Run the same configuration multiple times
- Verify that metrics remain consistent (within tolerance)

This aligns with the **non-functional requirement of reproducibility**

#### **7.3.4 Parameter Sensitivity Testing**

Evaluate system behavior under different configurations:

- Window size variations
- Batch size changes
- Different datasets (if available)

**Goal:**

- Understand impact on model performance
- Identify stable configurations

### **7.3.5 Performance Constraints**

The system must satisfy:

- Ability to process logs in batch mode
- Avoid memory overflow (controlled by sequence limits and batching)
- Stable execution without crashes

## **7.4 Deployment Plan**

### **7.4.1 Deployment Strategy**

Guardrail is deployed as a **local execution pipeline using Python CLI**.

**Execution Flow:**

1. User provides configuration (dataset path, checkpoint, parameters)
2. Pipeline is executed via script
3. Outputs are generated in specified directories

### **7.4.2 Deployment Requirements**

**Software Requirements:**

- Python environment
- Required ML libraries (e.g., PyTorch, Transformers)

**Hardware Requirements:**

- CPU for inference
- GPU (optional, for training/fine-tuning)

These align with the dependencies defined earlier

### 7.4.3 Execution Structure

Typical folder structure:

```
project/
|—input/
|—output/
|—configs/
|—models/
|—logs/
```

### 7.4.4 Output Artifacts

After execution, the system generates:

- Prediction files (CSV/JSON)
- Metrics report (Accuracy, Precision, Recall, F1)
- Execution logs

### 7.4.5 Reproducible Run Workflow

To ensure reproducibility:

1. Provide configuration file
2. Provide dataset and checkpoint
3. Execute pipeline
4. Compare output metrics

This corresponds to **Phase 6 (MVP packaging)** in implementation

## 7.5 Future Deployment Considerations (Optional Extension)

While the current system is batch-based, it can be extended to:

- Near real-time log processing
- Integration with SIEM tools
- Streaming pipelines (e.g., Kafka)

These are future improvements and not part of the current implementation.

## 7.6 Summary

The QA, performance, and deployment plan ensures that Guardrail:

- Produces correct and reproducible results
- Is robust against invalid inputs
- Provides measurable evaluation metrics
- Can be executed reliably using a configuration-driven pipeline

This makes the system suitable for both academic evaluation and practical SOC-oriented workflows.

## Chapter 8. Implementation Plan and Progress

### 8.1 Overview

This chapter describes the implementation plan and current progress of the Guardrail system. Guardrail is designed as a **reproducible, end-to-end log anomaly detection pipeline** that transforms raw logs into structured outputs through preprocessing, sequence modeling, anomaly detection, and evaluation.

The implementation follows a **phased approach**, ensuring that each component of the pipeline is validated before progressing to the next stage. The plan emphasizes correctness, reproducibility, and modular development.

### 8.2 Implementation Plan (Project-wide)

#### Phase 1: Project Initialization and Execution Readiness

**Goal:**

Ensure that the Guardrail system can be executed consistently across all team members.

**Activities:**

- Set up a standardized project structure (input, output, configs, logs)
- Configure Python environment and install dependencies
- Validate baseline script execution (no runtime errors)

**Completion Criteria:**

- All team members can run the project successfully
- A minimal pipeline execution completes without failure

#### Phase 2: Data Flow Validation (Logs → Sequences)

**Goal:**

Validate transformation of raw logs into model-ready sequences.

**Activities:**

- Implement and validate preprocessing:
  - Masking (timestamps, IDs, IPs)
  - Normalization of logs
- Implement sequence generation:
  - Sliding window

- Session window (if dataset supports)
- Validate:
  - Correct ordering of logs
  - No data leakage between train/test splits

### **Completion Criteria:**

- Consistent generation of training and testing datasets
- Verified sequence format and correctness

### **Phase 3: Model Execution Baseline (Inference + Metrics)**

#### **Goal:**

Ensure Guardrail can generate anomaly predictions and evaluation metrics.

#### **Activities:**

- Load model checkpoint
- Run inference on test sequences
- Generate predictions (Normal / Anomalous)
- Compute evaluation metrics:
  - Accuracy
  - Precision
  - Recall
  - F1-score

### **Completion Criteria:**

- Successful end-to-end run producing predictions and metrics
- Outputs saved in reusable format

### **Phase 4: Pipeline Hardening and Repeatability (WB2 Focus)**

#### **Goal:**

Improve system stability and ensure reproducibility of results.

#### **Activities:**

- Add input validation:
  - Missing dataset path
  - Invalid checkpoint path
- Standardize output formats:
  - Predictions

- Metrics
- Logs
- Run repeatability tests:
  - Same configuration → same results

### **Completion Criteria:**

- Stable execution without runtime failures
- Consistent outputs across multiple runs

## **Phase 5: Hybrid Detection Integration (Extension Based on Updated Scope)**

### **Goal:**

Extend Guardrail to support rule-based detection alongside ML-based anomaly detection.

### **Activities:**

- Define rule format (pattern-based rules)
- Implement rule evaluation module
- Run rule-based detection on log sequences
- Combine:
  - ML predictions
  - Rule-based detections

### **Completion Criteria:**

- System produces both anomaly predictions and rule matches
- Hybrid detection pipeline functions correctly

## **Phase 6: Contextual Output Generation (Extension Based on Updated Scope)**

### **Goal:**

Enhance interpretability by generating structured, investigation-ready outputs.

### **Activities:**

- Extract key entities:
  - IP addresses
  - Users
  - Processes (if present)
- Group related anomalies into patterns
- Generate structured summaries:
  - Why anomaly was detected

- Supporting evidence

**Completion Criteria:**

- Outputs include contextual explanations
- Results are interpretable for SOC workflows

**Phase 7: Benchmarking and Validation****Goal:**

Evaluate system robustness across configurations.

**Activities:**

- Test different parameters:
  - Window size
  - Batch size
- (Optional) Run on multiple datasets
- Compare performance metrics

**Completion Criteria:**

- Benchmark results documented
- Performance trends identified

**Phase 8: Final MVP Packaging****Goal:**

Prepare Guardrail for final submission and demonstration.

**Activities:**

- Finalize configuration files
- Document execution steps ("how to run")
- Package:
  - Code
  - Configs
  - Sample outputs
- Verify reproducibility

**Completion Criteria:**

- New user can run system end-to-end
- Final results reproducible

### 8.3 Progress Tracker

| Phase                         | Status                | Evidence                               |
|-------------------------------|-----------------------|----------------------------------------|
| Phase 1: Initialization       | Completed             | Setup notes, successful execution logs |
| Phase 2: Data Flow Validation | In Progress           | Sample sequences, processed outputs    |
| Phase 3: Baseline Execution   | Planned / In Progress | Prediction files, metrics              |
| Phase 4: Pipeline Hardening   | Planned               | Validation logs, repeatability results |
| Phase 5: Hybrid Detection     | Planned               | Rule definitions, detection outputs    |
| Phase 6: Contextual Output    | Planned               | Structured summaries                   |
| Phase 7: Benchmarking         | Planned               | Metrics comparison tables              |
| Phase 8: Final Packaging      | Planned               | Reproducible run guide                 |

### 8.4 Current Progress Summary

- Guardrail documentation for core chapters (literature, requirements, architecture, design) has been completed.
- The project codebase includes:
  - Preprocessing (masking and normalization)
  - Sequence generation (windowing)
  - Model inference and evaluation pipeline
- Initial setup and execution readiness are being validated.
- Next steps include:
  - Completing data flow validation
  - Running baseline inference and evaluation
  - Extending system with hybrid detection and contextual outputs

### 8.5 Risks and Limitations

- Availability of labeled datasets for evaluation
- Computational constraints for training/fine-tuning
- Variability in log formats across datasets
- Dependence on checkpoint compatibility

## 8.6 Summary

The implementation plan ensures that Guardrail is built in a structured and incremental manner, starting from execution readiness to full system integration. The phased approach supports:

- Reliable development
- Reproducible results
- Extension toward hybrid detection and contextual analysis

This aligns with both the **technical goals of the project** and the **updated scope for SOC-oriented functionality**.

## Chapter 9. Project Schedule

### Chapter 9: Project Schedule

#### 9.1 Overview

This section presents the project schedule for Guardrail, including **task assignments for each team member and timeline planning**. The schedule is organized into phases corresponding to the implementation plan and ensures clear ownership of responsibilities across the team

#### 9.2 Spring 2026 (Part A)

**Goal:** Foundation + pipeline validation + baseline execution + reproducibility

##### Milestone 1: Project Initialization & Execution Readiness

| Task No. | Task Description                                               | Team Member                   | Start Date | End Date |
|----------|----------------------------------------------------------------|-------------------------------|------------|----------|
| 1.1      | Create repo structure (inputs/outputs/configs/logs)            | Nikhil Dupally                | Mar 4      | Mar 5    |
| 1.2      | Set up Python environment and dependency installation          | Pooja Sindham Ramesh          | Mar 4      | Mar 6    |
| 1.3      | Validate baseline script execution (run test)                  | Vijaya Sharavan Reddy Badddam | Mar 4      | Mar 6    |
| 1.4      | Define configuration template (paths, window size, batch size) | Viswa Surya Kumar Suvvada     | Mar 4      | Mar 6    |

##### Milestone 2: Data Flow Validation (Logs → Sequences)

| Task No. | Task Description | Team Member | Start Date | End Date |
|----------|------------------|-------------|------------|----------|
|----------|------------------|-------------|------------|----------|

|     |                                                     |                                      |        |        |
|-----|-----------------------------------------------------|--------------------------------------|--------|--------|
| 2.1 | Implement preprocessing (masking + normalization)   | <b>Nikhil Dupally</b>                | Mar 11 | Mar 17 |
| 2.2 | Implement sliding window sequence generation        | <b>Pooja Ramesh Sindham</b>          | Mar 11 | Mar 20 |
| 2.3 | Implement session window generation (if applicable) | <b>Vijaya Sharavan Reddy Badddam</b> | Mar 18 | Mar 26 |
| 2.4 | Validate train/test split and prevent data leakage  | <b>Viswa Surya Kumar Suvvada</b>     | Mar 21 | Mar 31 |

### **Milestone 3: Baseline Model Execution (Inference + Metrics)**

| <b>Task No.</b> | <b>Task Description</b>                                      | <b>Team Member</b>                   | <b>Start Date</b> | <b>End Date</b> |
|-----------------|--------------------------------------------------------------|--------------------------------------|-------------------|-----------------|
| 3.1             | Select dataset and finalize configuration                    | <b>Nikhil Dupally</b>                | Apr 1             | Apr 2           |
| 3.2             | Run inference using model checkpoint                         | <b>Pooja Sindham Ramesh</b>          | Apr 2             | Apr 4           |
| 3.3             | Compute evaluation metrics (Accuracy, Precision, Recall, F1) | <b>Vijaya Sharavan Reddy Badddam</b> | Apr 3             | Apr 5           |
| 3.4             | Package outputs (predictions, metrics, logs)                 | <b>Viswa Surya Kumar Suvvada</b>     | Apr 5             | Apr 6           |

### **Milestone 4: Pipeline Hardening & Repeatability (WB2 Focus)**

| <b>Task No.</b> | <b>Task Description</b>                         | <b>Team Member</b>                   | <b>Start Date</b> | <b>End Date</b> |
|-----------------|-------------------------------------------------|--------------------------------------|-------------------|-----------------|
| 4.1             | Add input validation and error handling         | <b>Nikhil Dupally</b>                | Apr 7             | Apr 15          |
| 4.2             | Standardize output formats and folder structure | <b>Pooja Ramesh Sindham</b>          | Apr 7             | Apr 15          |
| 4.3             | Run repeatability tests and compare outputs     | <b>Vijaya Sharavan Reddy Badddam</b> | Apr 16            | Apr 25          |
| 4.4             | Build benchmarking table and summary analysis   | <b>Viswa Surya Kumar Suvvada</b>     | Apr 26            | May 1           |

#### **Milestone 5: Part A Consolidation**

| <b>Task No.</b> | <b>Task Description</b>                              | <b>Team Member</b>                   | <b>Start Date</b> | <b>End Date</b> |
|-----------------|------------------------------------------------------|--------------------------------------|-------------------|-----------------|
| 5.1             | Prepare reproducible run instructions ("how to run") | <b>Nikhil Dupally</b>                | May 2             | May 6           |
| 5.2             | Finalize configurations and dataset artifacts        | <b>Pooja Ramesh Sindham</b>          | May 2             | May 6           |
| 5.3             | Document limitations and risks                       | <b>Vijaya Sharavan Reddy Badddam</b> | May 7             | May 11          |
| 5.4             | Define roadmap for Part B                            | <b>Viswa Surya Kumar Suvvada</b>     | May 12            | May 15          |

### 9.3 Fall 2026 (Part B)

**Goal:** Extension (hybrid + contextual), benchmarking, optimization, final MVP

#### Milestone 6: Hybrid Detection & Contextual Layer Implementation

| Task No. | Task Description                                 | Team Member                   | Start Date | End Date |
|----------|--------------------------------------------------|-------------------------------|------------|----------|
| 6.1      | Define rule-based detection format and structure | Nikhil Dupally                | Aug 20     | Aug 30   |
| 6.2      | Implement rule-based detection module            | Pooja Ramesh Sindham          | Aug 20     | Aug 30   |
| 6.3      | Integrate ML + rule-based outputs                | Vijaya Sharavan Reddy Badddam | Aug 31     | Sep 12   |
| 6.4      | Implement contextual output generation           | Viswa Surya Kumar Suvvada     | Sep 13     | Sep 20   |

#### Milestone 7: Benchmarking & Robustness Evaluation

| Task No. | Task Description                                    | Team Member                   | Start Date | End Date |
|----------|-----------------------------------------------------|-------------------------------|------------|----------|
| 7.1      | Evaluate window size variations                     | Nikhil Dupally                | Sep 21     | Oct 5    |
| 7.2      | Evaluate preprocessing/masking strategies           | Pooja Ramesh Sindham          | Sep 21     | Oct 5    |
| 7.3      | Run evaluation on additional datasets (if feasible) | Vijaya Sharavan Reddy Badddam | Oct 6      | Oct 18   |

|     |                                            |                                  |        |        |
|-----|--------------------------------------------|----------------------------------|--------|--------|
| 7.4 | Generate benchmarking results and analysis | <b>Viswa Surya Kumar Suvvada</b> | Oct 19 | Oct 25 |
|-----|--------------------------------------------|----------------------------------|--------|--------|

#### **Milestone 8: Performance Optimization & Stability**

| <b>Task No.</b> | <b>Task Description</b>                            | <b>Team Member</b>                   | <b>Start Date</b> | <b>End Date</b> |
|-----------------|----------------------------------------------------|--------------------------------------|-------------------|-----------------|
| 8.1             | Optimize batching and memory usage                 | <b>Nikhil Dupally</b>                | Oct 26            | Nov 6           |
| 8.2             | Improve logging and reproducibility tracking       | <b>Pooja Ramesh Sindham</b>          | Oct 26            | Nov 6           |
| 8.3             | Perform regression testing and failure checks      | <b>Vijaya Sharavan Reddy Badddam</b> | Nov 7             | Nov 15          |
| 8.4             | Prepare performance report and readiness checklist | <b>Viswa Surya Kumar Suvvada</b>     | Nov 16            | Nov 20          |

#### **Milestone 9: Final MVP Packaging and Demo**

| <b>Task No.</b> | <b>Task Description</b>                                  | <b>Team Member</b>          | <b>Start Date</b> | <b>End Date</b> |
|-----------------|----------------------------------------------------------|-----------------------------|-------------------|-----------------|
| 9.1             | Package final pipeline (configs + outputs + checkpoints) | <b>Nikhil Dupally</b>       | Nov 21            | Nov 30          |
| 9.2             | Finalize documentation (architecture + usage guide)      | <b>Pooja Ramesh Sindham</b> | Nov 21            | Nov 30          |

|     |                                                    |                                      |       |        |
|-----|----------------------------------------------------|--------------------------------------|-------|--------|
| 9.3 | Execute final end-to-end validation and testing    | <b>Vijaya Sharavan Reddy Badddam</b> | Dec 1 | Dec 8  |
| 9.4 | Prepare demo presentation and submission materials | <b>Viswa Surya Kumar Suvvada</b>     | Dec 9 | Dec 15 |

## 9.4 Summary

The project schedule assigns **clear responsibilities to each team member** across all phases of development. This ensures:

- Balanced workload distribution
- Accountability for each component
- Smooth execution of the Guardrail pipeline

The schedule aligns with the implementation plan and supports the development of a **reproducible, hybrid, and context-driven log anomaly detection system**.

## Chapter 9: Project Schedule

### 9.1 Overview

This section presents the project schedule for Guardrail, including **task assignments for each team member and timeline planning**. The schedule is organized into phases corresponding to the implementation plan and ensures clear ownership of responsibilities across the team

### 9.2 Spring 2026 (Part A)

**Goal:** Foundation + pipeline validation + baseline execution + reproducibility

#### Milestone 1: Project Initialization & Execution Readiness

| Task No. | Task Description                                    | Team Member           | Start Date | End Date |
|----------|-----------------------------------------------------|-----------------------|------------|----------|
| 1.1      | Create repo structure (inputs/outputs/configs/logs) | <b>Nikhil Dupally</b> | Mar 4      | Mar 5    |

|     |                                                                |                                      |       |       |
|-----|----------------------------------------------------------------|--------------------------------------|-------|-------|
| 1.2 | Set up Python environment and dependency installation          | <b>Pooja Ramesh Sindham</b>          | Mar 4 | Mar 6 |
| 1.3 | Validate baseline script execution (run test)                  | <b>Vijaya Sharavan Reddy Badddam</b> | Mar 4 | Mar 6 |
| 1.4 | Define configuration template (paths, window size, batch size) | <b>Viswa Surya Kumar Suvvada</b>     | Mar 4 | Mar 6 |

### Milestone 2: Data Flow Validation (Logs → Sequences)

| Task No. | Task Description                                    | Team Member                          | Start Date | End Date |
|----------|-----------------------------------------------------|--------------------------------------|------------|----------|
| 2.1      | Implement preprocessing (masking + normalization)   | <b>Nikhil Dupally</b>                | Mar 11     | Mar 17   |
| 2.2      | Implement sliding window sequence generation        | <b>Pooja Ramesh Sindham</b>          | Mar 11     | Mar 20   |
| 2.3      | Implement session window generation (if applicable) | <b>Vijaya Sharavan Reddy Badddam</b> | Mar 18     | Mar 26   |
| 2.4      | Validate train/test split and prevent data leakage  | <b>Viswa Surya Kumar Suvvada</b>     | Mar 21     | Mar 31   |

### Milestone 3: Baseline Model Execution (Inference + Metrics)

| Task No. | Task Description | Team Member | Start Date | End Date |
|----------|------------------|-------------|------------|----------|
|----------|------------------|-------------|------------|----------|

|     |                                                              |                                      |       |       |
|-----|--------------------------------------------------------------|--------------------------------------|-------|-------|
| 3.1 | Select dataset and finalize configuration                    | <b>Nikhil Dupally</b>                | Apr 1 | Apr 2 |
| 3.2 | Run inference using model checkpoint                         | <b>Pooja Ramesh Sindham</b>          | Apr 2 | Apr 4 |
| 3.3 | Compute evaluation metrics (Accuracy, Precision, Recall, F1) | <b>Vijaya Sharavan Reddy Badddam</b> | Apr 3 | Apr 5 |
| 3.4 | Package outputs (predictions, metrics, logs)                 | <b>Viswa Surya Kumar Suvvada</b>     | Apr 5 | Apr 6 |

#### **Milestone 4: Pipeline Hardening & Repeatability (WB2 Focus)**

| <b>Task No.</b> | <b>Task Description</b>                         | <b>Team Member</b>                   | <b>Start Date</b> | <b>End Date</b> |
|-----------------|-------------------------------------------------|--------------------------------------|-------------------|-----------------|
| 4.1             | Add input validation and error handling         | <b>Nikhil Dupally</b>                | Apr 7             | Apr 15          |
| 4.2             | Standardize output formats and folder structure | <b>Pooja Ramesh Sindham</b>          | Apr 7             | Apr 15          |
| 4.3             | Run repeatability tests and compare outputs     | <b>Vijaya Sharavan Reddy Badddam</b> | Apr 16            | Apr 25          |
| 4.4             | Build benchmarking table and summary analysis   | <b>Viswa Surya Kumar Suvvada</b>     | Apr 26            | May 1           |

#### **Milestone 5: Part A Consolidation**

| <b>Task No.</b> | <b>Task Description</b>                              | <b>Team Member</b>                   | <b>Start Date</b> | <b>End Date</b> |
|-----------------|------------------------------------------------------|--------------------------------------|-------------------|-----------------|
| 5.1             | Prepare reproducible run instructions ("how to run") | <b>Nikhil Dupally</b>                | May 2             | May 6           |
| 5.2             | Finalize configurations and dataset artifacts        | <b>Pooja Ramesh Sindham</b>          | May 2             | May 6           |
| 5.3             | Document limitations and risks                       | <b>Vijaya Sharavan Reddy Badddam</b> | May 7             | May 11          |
| 5.4             | Define roadmap for Part B                            | <b>Viswa Surya Kumar Suvvada</b>     | May 12            | May 15          |

### 9.3 Fall 2026 (Part B)

**Goal:** Extension (hybrid + contextual), benchmarking, optimization, final MVP

#### Milestone 6: Hybrid Detection & Contextual Layer Implementation

| <b>Task No.</b> | <b>Task Description</b>                          | <b>Team Member</b>                   | <b>Start Date</b> | <b>End Date</b> |
|-----------------|--------------------------------------------------|--------------------------------------|-------------------|-----------------|
| 6.1             | Define rule-based detection format and structure | <b>Nikhil Dupally</b>                | Aug 20            | Aug 30          |
| 6.2             | Implement rule-based detection module            | <b>Pooja Ramesh Sindham</b>          | Aug 20            | Aug 30          |
| 6.3             | Integrate ML + rule-based outputs                | <b>Vijaya Sharavan Reddy Badddam</b> | Aug 31            | Sep 12          |

|     |                                        |                           |        |        |
|-----|----------------------------------------|---------------------------|--------|--------|
| 6.4 | Implement contextual output generation | Viswa Surya Kumar Suvvada | Sep 13 | Sep 20 |
|-----|----------------------------------------|---------------------------|--------|--------|

#### Milestone 7: Benchmarking & Robustness Evaluation

| Task No. | Task Description                                    | Team Member                   | Start Date | End Date |
|----------|-----------------------------------------------------|-------------------------------|------------|----------|
| 7.1      | Evaluate window size variations                     | Nikhil Dupally                | Sep 21     | Oct 5    |
| 7.2      | Evaluate preprocessing/masking strategies           | Pooja Ramesh Sindham          | Sep 21     | Oct 5    |
| 7.3      | Run evaluation on additional datasets (if feasible) | Vijaya Sharavan Reddy Badddam | Oct 6      | Oct 18   |
| 7.4      | Generate benchmarking results and analysis          | Viswa Surya Kumar Suvvada     | Oct 19     | Oct 25   |

#### Milestone 8: Performance Optimization & Stability

| Task No. | Task Description                             | Team Member          | Start Date | End Date |
|----------|----------------------------------------------|----------------------|------------|----------|
| 8.1      | Optimize batching and memory usage           | Nikhil Dupally       | Oct 26     | Nov 6    |
| 8.2      | Improve logging and reproducibility tracking | Pooja Ramesh Sindham | Oct 26     | Nov 6    |

|     |                                                    |                                      |        |        |
|-----|----------------------------------------------------|--------------------------------------|--------|--------|
| 8.3 | Perform regression testing and failure checks      | <b>Vijaya Sharavan Reddy Badddam</b> | Nov 7  | Nov 15 |
| 8.4 | Prepare performance report and readiness checklist | <b>Viswa Surya Kumar Suvvada</b>     | Nov 16 | Nov 20 |

### Milestone 9: Final MVP Packaging and Demo

| Task No. | Task Description                                         | Team Member                          | Start Date | End Date |
|----------|----------------------------------------------------------|--------------------------------------|------------|----------|
| 9.1      | Package final pipeline (configs + outputs + checkpoints) | <b>Nikhil Dupally</b>                | Nov 21     | Nov 30   |
| 9.2      | Finalize documentation (architecture + usage guide)      | <b>Pooja Ramesh Sindham</b>          | Nov 21     | Nov 30   |
| 9.3      | Execute final end-to-end validation and testing          | <b>Vijaya Sharavan Reddy Badddam</b> | Dec 1      | Dec 8    |
| 9.4      | Prepare demo presentation and submission materials       | <b>Viswa Surya Kumar Suvvada</b>     | Dec 9      | Dec 15   |

### 9.4 Summary

The project schedule assigns **clear responsibilities to each team member** across all phases of development. This ensures:

- Balanced workload distribution
- Accountability for each component
- Smooth execution of the Guardrail pipeline

The schedule aligns with the implementation plan and supports the development of a **reproducible, hybrid, and context-driven log anomaly detection system**.